Relazione Progetto TLN: Mazzei

Andrea Senese, Lorenzo Favaro, Lorenzo Vercelli

25 aprile 2022
Revisione: novembre 2026

Indice

1 Fase di Learning **3**
1.1 Matrice di transizione 3
1.2 Matrice di emissione 4
1.3 Strategie di smoothing 4

2 Fase di Decoding **6**
2.1 Most Frequent Tag (MFT) 6
2.2 Viterbi 6

3 Esperimenti **8**
3.1 Test set: test.conllu 10

Capitolo 1

Fase di Learning

La fase di learning si suddivide a sua volta in due sotto-fasi: la prima consiste nel costruire la matrice di transizione per calcolare le probabilità di transizione tra due tag qualsiasi, mentre nella seconda viene costruita la matrice di emissione contenente le singole probabilità di emissione di ogni parola dato l'insieme dei tag. Agli output di ciascuna di queste fasi è stata infine applicata la funzione logaritmica, per evitare l'underflow dovuto al prodotto di molte probabilità piccole. Per evitare il logaritmo di zero, prima di applicarlo viene sommato a ogni probabilità il più piccolo numero positivo rappresentabile (`np.finfo(float).tiny`).

1.1 Matrice di transizione

Per costruire ogni probabilità di transizione abbiamo utilizzato la libreria `conllu`. Tramite essa abbiamo effettuato il parsing delle frasi del training set, le quali vengono trasformate in liste di token, ossia dei dizionari contenenti le chiavi *form* (per indicare la parola) e *lemma* (per indicare il NER-tag associato).

Dati due tag sequenziali t_{i-1} e t_i, abbiamo creato i dizionari `tag_counter` per tenere il conto delle occorrenze di ogni tag (ossia $C(t_{i-1})$) e `transition_counter` per contare le occorrenze della coppia (ossia $C(t_{i-1}, t_i)$). Innanzitutto, per ogni frase parsificata contiamo una volta il tag fittizio *START* e la transizione da *START* al primo tag della frase. Dopodiché, generiamo tutte le coppie sequenziali fino al penultimo token e incrementiamo di uno i valori delle relative chiavi dei due dizionari. Ad esempio, data la seguente frase:

Paolo\B-PER ama\O Francesca\B-PER dolcemente\O

le coppie di tag ricavate sono `[(B-PER,O), (O,B-PER), (B-PER,O)]`. L'ultimo tag della frase non viene conteggiato in `tag_counter`, in quanto non ha alcuna transizione uscente: in questo modo il denominatore di ogni probabilità conta solo le occorrenze del tag da cui parte effettivamente una transizione, e ogni riga della matrice somma a 1.

Una volta che i dizionari sono stati popolati, per calcolare la probabilità di transizione $P(t_i \mid t_{i-1})$ abbiamo generato tutte le combinazioni di tag e in corrispondenza della riga t_{i-1} e della colonna t_i inseriamo il valore `transition_counter[(`t_{i-1}`, `t_i`)] / tag_counter[`t_{i-1}`]`. La riga *START* costituisce il vettore Π delle probabilità iniziali. Di seguito la matrice ottenuta sul training set inglese:

	O	I-LOC	I-MISC	I-ORG	I-PER	B-LOC	B-MISC	B-PER	B-ORG
START	0.92	0.00	0.00	0.00	0.00	0.02	0.01	0.04	0.01
O	0.92	0.00	0.00	0.00	0.00	0.03	0.02	0.02	0.01
I-LOC	0.75	0.24	0.00	0.00	0.00	0.00	0.00	0.00	0.00
I-MISC	0.46	0.00	0.54	0.00	0.00	0.00	0.00	0.00	0.00
I-ORG	0.59	0.00	0.00	0.41	0.00	0.00	0.00	0.00	0.00
I-PER	0.88	0.00	0.00	0.00	0.12	0.00	0.00	0.00	0.00
B-LOC	0.74	0.26	0.00	0.00	0.00	0.00	0.00	0.00	0.00
B-MISC	0.65	0.00	0.32	0.00	0.00	0.00	0.02	0.00	0.00
B-PER	0.36	0.00	0.00	0.00	0.64	0.00	0.00	0.00	0.00
B-ORG	0.50	0.00	0.00	0.49	0.00	0.00	0.00	0.00	0.00

1.2 Matrice di emissione

La costruzione della matrice di emissione passa attraverso due step principali: il primo consiste nel computare una volta sola le probabilità di emissione per le parole presenti nel training set e il secondo nel costruire la matrice vera a propria per ogni frase del test set, impiegando eventuali strategie di smoothing nel caso in cui una parola non sia presente nel training set. Nel primo step abbiamo utilizzato un nuovo dizionario `word_tag_counter` per contare le occorrenze di ogni coppia (w_i,t_i) (parola-tag), oltre a quello già utilizzato precedentemente `tag_counter`. La probabilità di emissione $P(w_i \mid t_i)$ è stata quindi calcolata come `word_tag_counter[(`w_i`,`t_i`)] / tag_counter[`t_i`]`.

Per quanto riguarda il secondo step, abbiamo definito una matrice le cui righe sono i tag e le cui colonne sono le parole delle frasi del test set. Qui utilizziamo l'output dello step precedente per generare tutte le combinazioni di coppie (parola-tag) per produrre le probabilità di emissione da associare agli elementi della matrice. Distinguiamo due casi:

- se la parola compare nel training set (`vocabulary`), le viene assegnata la probabilità di emissione calcolata; se la parola non è mai stata osservata con un determinato tag, la probabilità per quel tag è 0;
- se la parola non compare nel training set, la probabilità di emissione per ogni tag si ottiene utilizzando una delle strategie di smoothing descritte nella prossima sezione.

È importante che lo smoothing venga applicato solo alle parole sconosciute: applicandolo anche alle coppie parola-tag mai osservate di parole note, queste riceverebbero probabilità molto più alte di quelle reali (ad esempio $\frac{1}{9}$ con lo smoothing uniforme), compromettendo la classificazione anche delle parole note.

Di seguito la matrice di emissione (senza smoothing) per una frase del test set inglese:

	She	proceeded	to	the	University	of	the	Philippines	College	of	Law	.
O	0.00	0.00	0.02	0.06	0.00	0.03	0.06	0.00	0.00	0.03	0.00	0.05
I-LOC	0.00	0.00	0.00	0.00	0.00	0.04	0.00	0.00	0.00	0.04	0.00	0.00
I-MISC	0.00	0.00	0.01	0.03	0.00	0.05	0.03	0.00	0.00	0.05	0.00	0.00
I-ORG	0.00	0.00	0.00	0.01	0.02	0.06	0.01	0.00	0.01	0.06	0.00	0.00
I-PER	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
B-LOC	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
B-MISC	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
B-PER	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
B-ORG	0.00	0.00	0.00	0.00	0.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00

1.3 Strategie di smoothing

Le strategie di smoothing che abbiamo implementato sono raccolte in una classe enumeratore "SmoothingStrategy" utile a specificare quale venga utilizzata al momento dell'e-

secuzione. Tutte le strategie si applicano esclusivamente alle parole w_i assenti dal training set. In questa sezione ci concentriamo sul descrivere l'ultima strategia di quelle elencate di seguito, in quanto l'implementazione delle altre è immediata.

1. `uniform`: $P(w_i \mid t_i) = \frac{1}{\#NER_TAGs}$
2. `always_other`: $P(w_i \mid \texttt{O}) = 1$
3. `other_and_misc`: $P(w_i \mid \texttt{O}) = P(w_i \mid \texttt{B_MISC}) = 0.5$
4. `one_shot_word`: $P(w_i \mid t_i) = \frac{C(w_i,t_i)}{\sum_{j \in len(T)} C(w_j,t_j)}$

Per l'implementazione di `one_shot_word` abbiamo utilizzato come dataset di riferimento il development set (`val.conllu`). Abbiamo popolato il dizionario `word_tag_counter`, per poi andare a calcolare per ogni tag t_i la distribuzione delle probabilità di emissione di tutte le coppie (w_i, t_i) la cui occorrenza è pari a 1. Quindi, durante la computazione della matrice di emissione, ad ogni parola sconosciuta verrà assegnata, per ciascun tag, la probabilità associata al tag calcolata rispetto alle parole con occorrenza singola.

Capitolo 2

Fase di Decoding

Nella fase di decoding, per ogni frase viene rilevato l'insieme dei tag reali (reference), assieme alla relativa matrice di emissione. Quindi, in base all'algoritmo di decoding selezionato precedentemente viene costruito l'insieme dei tag predetti dall'algoritmo. I risultati e le reference vengono quindi inseriti in due liste globali utilizzate per calcolare la bontà degli algoritmi. Le metriche impiegate per la valutazione sono state importate da Scikit Learn. Nelle seguenti sezioni vengono discussi gli algoritmi di decoding implementati.

2.1 Most Frequent Tag (MFT)

L'algoritmo "Most Frequent Tag" è una delle baseline implementate e restituisce come predizione la lista di tag più frequenti rispetto a una data sequenza di parole in input, senza tenere conto del contesto. Nello specifico, ad ogni parola viene associato il tag con maggior probabilità $P(t \mid w)$. Per il teorema di Bayes $P(t \mid w) \propto P(w \mid t) \cdot P(t)$, perciò la predizione si ottiene sommando (in scala logaritmica) alla colonna della matrice di emissione il vettore `tag_distrib`, contenente la probabilità a priori $P(t)$ di ciascun tag calcolata sul training set:

$$\hat{t} = \arg\max_t \left[\log P(w \mid t) + \log P(t)\right]$$

Usare la sola $P(w \mid t)$ favorirebbe i tag rari: dividendo per il numero di occorrenze del tag, le probabilità di emissione del tag `O`, molto frequente, risulterebbero sistematicamente più basse.

2.2 Viterbi

L'implementazione dell'algoritmo di Viterbi segue da vicino quella vista a lezione, e fa uso di una matrice di Viterbi inizializzata a $-\infty$ (per via del passaggio ai logaritmi). È suddivisa in tre punti principali:

1. le probabilità iniziali vengono calcolate sommando (e non moltiplicando, per via dei logaritmi) i valori della prima colonna della matrice di emissione a quelli dell'array Π (contenente i valori delle probabilità di transizione dal tag fittizio START ad ogni altro);

2. per il calcolo delle probabilità intermedie abbiamo ridefinito la funzione *argmax* in modo che restituisca sia l'indice del tag più frequente, utilizzato per il calcolo del backpointer, sia il suo valore di probabilità di transizione;

3. per l'ultima parola è sufficiente applicare l'argmax all'ultima colonna della matrice di viterbi e restituire l'indice del tag, che viene utilizzato come primo punto di partenza nella matrice backpointer per la ricostruzione del percorso migliore.

Capitolo 3

Esperimenti

Di seguito vengono discussi alcuni esperimenti che abbiamo effettuato nel corso dell'esercitazione, con le relative osservazioni. Abbiamo eseguito entrambi gli algoritmi di decoding applicando le varie strategie di smoothing implementate, addestrando il modello sul training set italiano.

Il primo esperimento è stato eseguito sulla frase "La vera casa di Harry Potter è il castello di Hogwarts".

	La	**vera**	**casa**	**di**	**Harry**	**Potter**	**è**	**il**	**castello**	**di**	**Hogwarts**	**.**
Reference	O	O	O	O	B-MISC	I-MISC	O	O	O	O	B-MISC	O

I risultati ottenuti sono i seguenti:

Viterbi

	La	**vera**	**casa**	**di**	**Harry**	**Potter**	**è**	**il**	**castello**	**di**	**Hogwarts**	**.**
No-smoothing	O	O	O	O	B-MISC	I-MISC	O	O	O	O	B-MISC	O
Uniform	O	O	O	O	B-MISC	I-MISC	O	O	O	O	B-MISC	O
Always-other	O	O	O	O	B-MISC	I-MISC	O	O	O	O	B-MISC	O
Other-and-Misc	O	O	O	O	B-MISC	I-MISC	O	O	O	O	B-MISC	O
Oneshot-Word	O	O	O	O	B-MISC	I-MISC	O	O	O	O	B-MISC	O

Viterbi assegna il tag corretto a ogni parola, indipendentemente dalla strategia di smoothing: tutte le parole della frase compaiono nel training set, quindi lo smoothing non entra in gioco.

MFT

	La	**vera**	**casa**	**di**	**Harry**	**Potter**	**è**	**il**	**castello**	**di**	**Hogwarts**	**.**
No-smoothing	O	O	O	O	B-PER	I-MISC	O	O	O	O	B-MISC	O
Uniform	O	O	O	O	B-PER	I-MISC	O	O	O	O	B-MISC	O
Always-other	O	O	O	O	B-PER	I-MISC	O	O	O	O	B-MISC	O
Other-and-Misc	O	O	O	O	B-PER	I-MISC	O	O	O	O	B-MISC	O
Oneshot-Word	O	O	O	O	B-PER	I-MISC	O	O	O	O	B-MISC	O

La baseline (MFT), non tenendo conto del contesto, sbaglia il tag di "Harry": da sola la parola è più frequentemente il nome di una persona (`B-PER`), mentre qui fa parte del titolo "Harry Potter" (`B-MISC`). In termini di accuracy abbiamo riscontrato i seguenti risultati:

	Viterbi	**MFT**
No-smoothing	1.00	0.92
Uniform	1.00	0.92
Always-other	1.00	0.92
Other-and-Misc	1.00	0.92
Oneshot-Word	1.00	0.92

Rispetto alla baseline implementata, le performance dell'algoritmo di Viterbi sono risultate migliori in tutti i casi.

Di seguito riportiamo l'elenco degli ulteriori esperimenti effettuati con annessi risultati.

Secondo esperimento: "Harry le raccontò del loro incontro a Diagon Alley."

	Harry	le	raccontò	del	loro	incontro	a	Diagon	Alley	.
Reference	B-MISC	O	O	O	O	O	O	B-LOC	I-LOC	O

Viterbi

	Harry	le	raccontò	del	loro	incontro	a	Diagon	Alley	.
No-smoothing	B-PER	O	O	O	O	O	O	B-PER	I-PER	O
Uniform	B-PER	O	O	O	O	O	O	B-PER	I-PER	O
Always-other	B-PER	O	O	O	O	O	O	O	O	O
Other-and-Misc	B-PER	O	O	O	O	O	O	B-MISC	I-PER	O
Oneshot-Word	B-PER	O	O	O	O	O	O	B-PER	I-PER	O

MFT

	Harry	le	raccontò	del	loro	incontro	a	Diagon	Alley	.
No-smoothing	B-PER	O	O	O	O	O	O	O	I-PER	O
Uniform	B-PER	O	O	O	O	O	O	O	I-PER	O
Always-other	B-PER	O	O	O	O	O	O	O	I-PER	O
Other-and-Misc	B-PER	O	O	O	O	O	O	O	I-PER	O
Oneshot-Word	B-PER	O	O	O	O	O	O	O	I-PER	O

In questa frase "Harry" indica la persona, per cui il tag `B-PER` predetto da entrambi gli algoritmi è da considerarsi corretto, nonostante la reference riporti `B-MISC`. "Diagon Alley" invece non viene riconosciuto come luogo: "Diagon" è assente dal training set, mentre "Alley" vi compare soltanto con il tag `I-PER`; per questo, senza smoothing, Viterbi riconosce l'entità ma la classifica come persona.

Terzo esperimento: "Mr Dursley era direttore di una ditta di nome Grunnings, che fabbricava trapani."

	Mr	Dursley	era	direttore	di	una	ditta	di	nome	Grunnings	,	che	fabbricava	trapani	.
Reference	B-MISC	I-MISC	O	O	O	O	O	O	O	B-ORG	O	O	O	O	O

Viterbi

	Mr	Dursley	era	direttore	di	una	ditta	di	nome	Grunnings	,	che	fabbricava	trapani	.
No-smoothing	B-MISC	I-MISC	O	O	O	O	O	O	O	O	O	O	O	O	O
Uniform	B-MISC	I-MISC	O	O	O	O	O	O	O	O	O	O	O	O	O
Always-other	B-MISC	O	O	O	O	O	O	O	O	O	O	O	O	O	O
Other-and-Misc	B-MISC	O	O	O	O	O	O	O	O	O	O	O	O	O	O
Oneshot-Word	B-MISC	O	O	O	O	O	O	O	O	O	O	O	O	O	O

MFT

	Mr	Dursley	era	direttore	di	una	ditta	di	nome	Grunnings	,	che	fabbricava	trapani	.
No-smoothing	B-MISC	O	O	O	O	O	O	O	O	O	O	O	O	O	O
Uniform	B-MISC	O	O	O	O	O	O	O	O	O	O	O	O	O	O
Always-other	B-MISC	O	O	O	O	O	O	O	O	O	O	O	O	O	O
Other-and-Misc	B-MISC	O	O	O	O	O	O	O	O	O	O	O	O	O	O
Oneshot-Word	B-MISC	O	O	O	O	O	O	O	O	O	O	O	O	O	O

Nessuna configurazione riconosce "Grunnings", parola assente dal training set, come organizzazione. Senza smoothing (o con smoothing uniforme) Viterbi riconosce "Mr Dursley" come un'unica entità grazie alle transizioni, mentre con le altre strategie la parola sconosciuta "Dursley" riceve il tag `O`, spezzando l'entità.

3.1 Test set: test.conllu

Abbiamo testato gli algoritmi di Viterbi e di baseline sul test-set italiano e inglese, con tutte le strategie di smoothing. Le accuracy ottenute sono le seguenti:

	Viterbi ENG	**MFT ENG**	**Viterbi ITA**	**MFT ITA**
No-smoothing	96.31%	94.35%	97.44%	95.75%
Uniform	96.31%	94.35%	97.44%	95.75%
Always-other	95.73%	94.35%	97.06%	95.75%
Other-and-Misc	95.77%	94.35%	97.06%	95.75%
Oneshot-Word	95.94%	94.35%	97.13%	95.75%

Viterbi supera la baseline in tutte le configurazioni, di circa 1,5–2 punti percentuali. Lo smoothing invece non migliora le prestazioni: le parole sconosciute sono poche (circa il 3% dei token del test set inglese) e la strategia uniforme equivale a non applicare smoothing, perché assegna la stessa probabilità a ogni tag e lascia decidere le sole transizioni. Sulla baseline lo smoothing non ha alcun effetto, perché per le parole sconosciute il tag predetto resta `O` in ogni caso.

Va osservato che l'accuracy è fortemente influenzata dallo sbilanciamento delle classi: il tag `O` copre circa l'88% dei token, per cui anche un sistema che predicesse sempre `O` otterrebbe un'accuracy simile. Per questo motivo riportiamo anche precision, recall e f1-score per ciascun tag, dai quali si osserva che sui tag delle entità le prestazioni sono sensibilmente più basse.

Di seguito i risultati completi di Viterbi senza smoothing.

Parametri:

- Algoritmo: Viterbi;
- Smoothing: None;
- Dataset: Inglese;

```
Total words: 267155
Accuracy: 96.31%

Classification Report
              precision    recall  f1-score   support

           O       0.98      1.00      0.99    234498
       I-LOC       0.87      0.75      0.80      2063
      I-MISC       0.79      0.64      0.71      4897
       I-ORG       0.82      0.76      0.79      2622
       I-PER       0.91      0.84      0.87      3959
       B-LOC       0.83      0.76      0.79      5955
      B-MISC       0.72      0.64      0.68      4505
       B-PER       0.89      0.77      0.83      5207
       B-ORG       0.80      0.70      0.75      3449

    accuracy                           0.96    267155
   macro avg       0.85      0.76      0.80    267155
weighted avg       0.96      0.96      0.96    267155

Confusion Matrix
             O   I-LOC  I-MISC   I-ORG   I-PER   B-LOC  B-MISC   B-PER   B-ORG
O       233491      37     272      97      77      98     228      74     124
I-LOC      206    1541      55      89      43      92      15      14       8
I-MISC     865      44    3110     112      61      91     499      55      60
I-ORG      241      84     104    1983      33      56      22      13      86
I-PER      428      13      47      10    3311      30       6     109       5
B-LOC      955      43      26      17      30    4528     108      77     171
B-MISC     951       7     238      14       7     178    2894     109     107
B-PER      771       5      30       1      83     135     111    4032      39
B-ORG      482       3      32      82       7     245     135      49    2414
```

Parametri:

- Algoritmo: Viterbi;
- Smoothing: None;
- Dataset: Italiano;

```
Total words: 313066
Accuracy: 97.44%

Classification Report
              precision    recall  f1-score   support

           O       0.98      1.00      0.99    275826
       I-LOC       0.86      0.73      0.79      3034
      I-MISC       0.79      0.62      0.69      3331
       I-ORG       0.89      0.75      0.81      1045
       I-PER       0.94      0.89      0.91      7094
       B-LOC       0.90      0.83      0.87      9788
      B-MISC       0.81      0.66      0.73      2342
       B-PER       0.95      0.86      0.90      8377
       B-ORG       0.88      0.83      0.85      2229

    accuracy                           0.97    313066
   macro avg       0.89      0.80      0.84    313066
weighted avg       0.97      0.97      0.97    313066

Confusion Matrix
             O   I-LOC  I-MISC   I-ORG   I-PER   B-LOC  B-MISC   B-PER   B-ORG
O       274876     162     309      16     104     187      77      72      23
I-LOC      472    2204      48      22      70     165      15      34       4
I-MISC     888      45    2052      33      63     126      78      33      13
I-ORG      139      26      47     781       7      29      11       1       4
I-PER      476      25      25       9    6333      96      15     107       8
B-LOC     1288      86      13       2      31    8170      34      60     104
B-MISC     528       2      69       1       6      84    1538      57      57
B-PER      730      26      22       0     146     107      65    7239      42
B-ORG      206       1      19      10       5      65      59      19    1845
```